第一部分：在现代 CPU 上的性能分析

2 性能测量

了解应用程序性能的第一步是知道如何进行测量。人们将性能视为应用程序的特性之一。[8] 但与其他特性不同，性能不是一个布尔属性：一个应用程序可以非常慢，飞快，或者介于两者之间。这就是为什么对于是否具有性能的问题，不可能回答“是”或“否”的原因。

性能问题通常比大多数功能问题更难追踪和复现。尽管有时我们不得不处理非确定性的，难以复现的性能错误。通常，程序的每次运行在功能上都是相同的，但从性能的角度来看有些许差异。例如，解压缩 zip 文件时，我们一遍又一遍地获得相同的结果，这意味着这个操作是可重现的。然而，要精确复现此操作的每个 CPU 周期的性能特征是不可能的。

任何关心性能评估的人都知道有时进行公正的性能测量并从中得出准确的结论是多么困难。性能测量可能非常出乎意料和反直觉。更改源代码中看似无关的部分可能会让我们惊讶地发现对程序性能有重大影响。这种现象被称为测量偏差。由于测量中存在误差，性能分析需要使用统计方法来处理它们。这个主题值得一整本书来单独讨论。在这里，我们只关注高层次的思想和需要遵循的方向。

进行公平的性能实验是获得准确而有意义的结果的关键步骤。设计性能测试和配置环境都是评估性能过程中的重要组成部分。本章将简要介绍现代系统为何产生嘈杂的性能测量以及如何处理这种情况。我们将涉及在实际生产部署中测量性能的重要性。

没有一个长寿命的产品是不会发生性能退化的。这对于有许多贡献者的大型项目尤为重要，因为变化发生得非常迅速。本章将花几页讨论在持续集成和持续交付（CI/CD）系统中跟踪性能变化的自动化过程。我们还提供有关开发人员在源代码库中实施更改时如何正确收集和分析性能测量的一般指导。

本章的结尾描述了开发人员在基于时间的测量中可以使用的软件和硬件定时器，以及在设计和编写良好的微基准时可能遇到的常见问题。

2.1 现代系统中的噪声

硬件和软件中有许多旨在提高性能的特性。但并非所有特性都具有确定性行为。让我们考虑动态频率调整[9]（DFS）：这是一种允许 CPU 在短时间内增加频率的特性，从而使其运行速度显著提高。然而，CPU 不能长时间保持在“超频”模式下，因此稍后会将频率降回基础值。DFS 通常在很大程度上取决于核心温度，这使得很难预测对我们实验的影响。

如果我们在“冷”处理器上连续启动两次基准测试，[10]第一次运行可能会在一段时间内以“超频”模式运行，然后将频率降回基准水平。然而，第二次运行可能没有这种优势，将以基准频率运行而不进入“Turbo 模式”。尽管我们两次运行了完全相同版本的程序，但它们运行的环境并不相同。图 2 显示了动态频率调整可能导致测量结果的差异。正如你所看到的，由于第一次运行在更高的频率下运行，因此比第二次运行快 1 秒。在笔记本电脑上进行基准测试时，这样的情况经常发生，因为它们的散热能力有限。

频率调整是一种硬件特性，但测量中的变化也可能来自软件特性。让我们考虑一个文件系统缓存的例子。如果我们

[8] Nelson Elhage 的博客文章“Reflections on software performance”: https://blog.nelhage.com/post/reflections-on-performance/。

[9] 动态频率缩放 - https://en.wikipedia.org/wiki/Dynamic_frequency_scaling

[10] 冷处理器是指处于空闲模式一段时间，使其冷却下来的 CPU

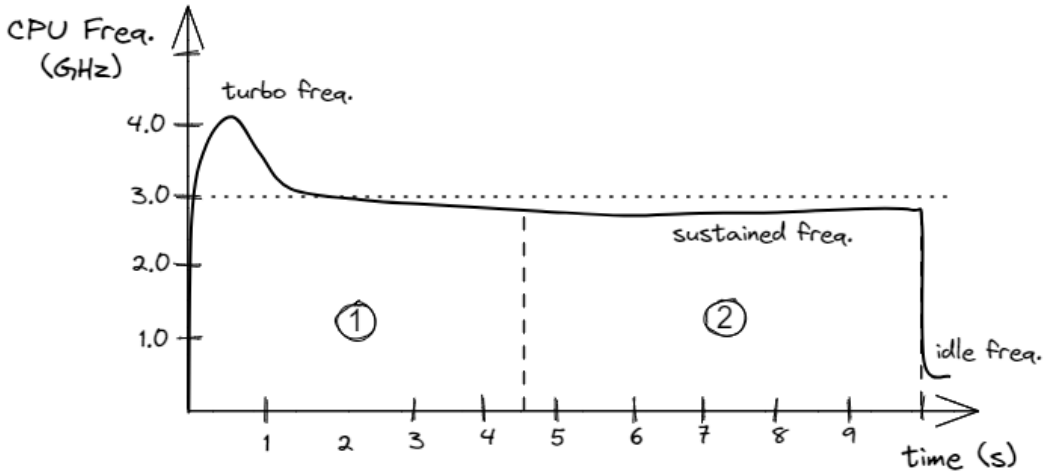


Figure 2: 频率缩放导致性能变化的差异：第一次运行比第二次运行快 1 秒。

对进行大量文件操作的应用程序进行基准测试，例如`git status`命令，文件系统可能在性能方面起到重要作用。当运行基准测试的第一次迭代时，文件系统缓存中可能缺少所需的条目。然而，当第二次运行相同的基准测试时，文件系统缓存将被热身，使其比第一次运行明显更快。

不幸的是，测量偏差不仅来自环境配置。[Mytkowicz et al., 2009] 的论文表明，UNIX 环境大小（即存储环境变量所需的总字节数）和链接顺序（给链接器提供的目标文件的顺序）可能以不可预测的方式影响性能。此外，还有许多其他影响内存布局并可能影响性能测量的方法。[Curtsinger & Berger, 2013] 提出了一种在现代架构上实现统计上可靠的软件性能分析的方法。这项工作表明，通过在运行时高效且重复地随机化代码、堆栈和堆对象的放置，可以消除来自内存布局的测量偏差。不幸的是，这些想法并没有得到很大发展，现在这个项目几乎被放弃了。

请记住，即使运行类似 Linux top 这样的任务管理工具也会影响测量，因为 CPU 核心会被激活并分配给它。这可能会影响实际基准测试运行的核心频率。

要获得一致的测量结果，需要以相同的条件运行所有基准测试的所有迭代。然而，要复制完全相同的环境并完全消除偏差是不可能的：可能会存在不同的温度条件、电力传递峰值、运行的邻近进程等。追踪系统中所有潜在的噪音和变化可能是一个永无止境的故事。有时无法实现，例如在对大型分布式云服务进行基准测试时。

为了确保性能测试（例如微基准测试）具有明确且稳定的结果，消除系统中的非确定性行为是有益的。例如，当您实现代码更改并希望通过对相同程序的两个不同版本进行基准测试来了解相对加速比时，您可以控制基准测试的大多数变量，包括其输入、环境配置等。在这种情况下，消除系统中的非确定性有助于获得更加一致和准确的比较。完成本地测试后，请记住验证投影性能改进是否反映在实际测量中。读者可以在附录 A 中找到一些示例，这些示例说明了哪些功能会使性能测量产生噪音以及如何禁用它们。此外，还有一些工具可以设置环境以确保基准测试结果具有低方差，其中一个这样的工具是 temci[11]。

在评估实际性能提升时，不建议消除系统非确定性行为。工程师应该尝试复制他们正在优化的目标系统配置。对测试系统进行任何人工调整都会使结果与您的服务用户在实际中看到的结果相背离。此外，任何性能分析工作，包括分析（参见 Section 5.5），都应在与实际部署中类似的系统上进行。

最后，重要的是要记住，即使特定硬件或软件功能具有非确定性行为，也并不意味着它被认为是有害的。它可能会产生不一致的结果，但它旨在提高系统的整体性能。禁用此类功能可能会减少微基准测试中的噪音，但会使整个套件运行更长时间。这对于 CI/CD 性能测试尤其重要，因为整个基准测试套件的运行时间有限制。

[11] Temci - https://github.com/parttimenerd/temci

2.2 在生产环境中进行性能测量

当一个应用程序在共享基础设施上运行时（在公共云中很常见），通常会有来自其他客户的其他工作负载在同一台服务器上运行。随着虚拟化和容器等技术变得越来越流行，公共云提供商试图充分利用他们服务器的容量。不幸的是，这为在这样的环境中进行性能测量带来了额外的障碍。与相邻进程共享资源可能以不可预测的方式影响性能测量。

通过在实验室中重现生产工作负载来分析生产工作负载可能会很棘手。有时，对于“内部”性能测试，可能无法合成精确的行为。这就是为什么越来越多的云提供商和超大规模云服务提供商选择直接在生产系统上进行性能分析和监控的原因。在“没有其他参与者”的情况下进行性能测量可能不反映真实的场景。在实验室环境中执行性能优化代码可能会得到良好的性能表现，但在生产环境中可能不尽如人意。话虽如此，这并不意味着不需要持续的“内部”测试来及早捕捉性能问题。并非所有性能回归都可以在实验室中捕捉到，但工程师应设计具有代表性的真实场景性能基准测试。

大型服务提供商实施监控用户设备上性能的遥测系统正在成为一种趋势。Netflix Icarus[12]遥测服务就是一个例子，它在全球范围内的数千台不同设备上运行。这种遥测系统帮助 Netflix 了解真实用户如何感知 Netflix 应用的性能。它允许工程师分析从许多设备收集的数据，并找到否则无法发现的问题。这种数据可以帮助更明智地决定在哪里集中优化工作。

监控生产部署的一个重要注意事项是测量开销。因为任何类型的监控都会影响正在运行的服务的性能，因此建议仅使用轻量级的分析方法。根据 [Ren et al., 2010] 的说法：“对于提供实时流量的数据中心机器进行持续性能分析，极低的开销至关重要”。通常，可接受的聚合开销被认为低于 1%。性能监控开销可以通过限制受监控机器的集合以及使用更长的时间间隔来减少。

在这样的生产环境中进行性能测量意味着我们必须接受其嘈杂的特性，并使用统计方法来分析结果。[Liu et al., 2019] 提供了一个好例子，说明像 LinkedIn 这样的大公司如何在生产环境中使用统计方法来测量和比较基于分位数的指标（例如 90 分位数的页面加载时间）在其 A/B 测试中的表现。

2.3 自动检测性能回归

软件供应商试图增加部署频率正在成为一种趋势。公司不断寻求加速产品上市的方式。不幸的是，这并不意味着每次新发布的软件产品都会变得更好。特别是，软件性能缺陷往往以惊人的速度泄漏到生产软件中 [Jin et al., 2012]。在软件的演变过程中，大量的变化给分析所有这些结果和历史数据以检测性能回归带来了挑战。

软件性能回归是在软件从一个版本发展到另一个版本时错误地引入的缺陷。捕捉性能错误和改进意味着检测哪些提交改变了软件的性能（由性能测试测量），在测试基础设施的噪声存在的情况下。从数据库系统到搜索引擎再到编译器，几乎所有大规模软件系统在其持续演进和部署生命周期中都会经历性能回归。在软件开发过程中完全避免性能回归可能是不可能的，但通过适当的测试和诊断工具，可以显着减少这些缺陷悄然泄漏到生产代码中的可能性。

首先考虑的选择是：让人类查看图表并比较结果。毫不奇怪，我们很快就想要摆脱这个选择。人们往往很快失去注意力，可能会错过回归，特别是在嘈杂的图表上，例如图 3 所示的图表。人类可能会捕捉到发生在 8 月 5 日左右的性能回归，但人们可能不会发现后续的回归。除了容易出错外，让人类参与其中还是一项耗时且枯燥的工作，必须每天进行。

第二个选择是设置一个阈值，例如 2%：认为性能在阈值内的每个代码修改都是噪声，而超过阈值的则被认为是回归。这比第一个选择要好一些，但仍然有其自身的缺点。性能测试中的波动是不可避免的：有时，甚至一个无害的代码更改[13]也可能触发基准测试中的性能变化。选择正确的阈值极其困难，也不能保证低误报率和低漏报率。将阈值设置得太低可能会导致分析一堆并非由源代码更改引起而是由某些随机噪声引起的小型回归。将阈值设置得太高可能会过滤掉真正的性能回归。小的变化可能会逐渐积累成较大的回归，这可能会被忽视。例如，假设您设置了 2%

[12] 在 CMG 2019 年展示，https://www.youtube.com/watch?v=4RG2DUK03_0。

[13] 下面的文章表明，改变函数的顺序或删除死函数可能会导致性能变化: https://easyperf.net/blog/2018/01/18/Code_alignment_issues

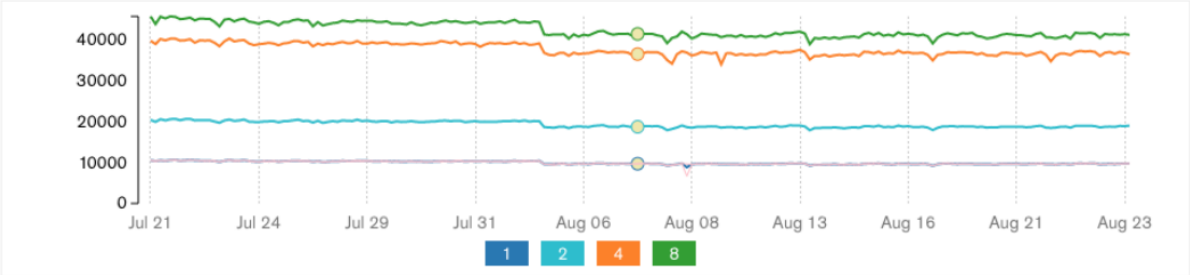


Figure 3: 4 个测试的性能趋势图，8 月 5 日性能略有下降 (值越高越好). © Image from [Daly et al., 2020]

的阈值。如果有两次连续的 1.5% 的回归，它们都将被过滤掉。但是在两天内，性能回归将累积达到 3%，这超过了阈值。通过观察图 3 ，我们可以得出一个观察结果，即阈值需要进行每个测试的调整。对于绿色（上线）测试有效的阈值未必对于紫色（下线）测试同样有效，因为它们具有不同级别的噪声。每个测试都需要设置明确的阈值值以警报回归的 CI 系统示例是LUCI[14]，它是 Chromium 项目的一部分。

第三个选择是使用统计分析来识别性能回归。一个简单的例子是使用学生 t 检验[15]来比较程序 A 的 100 次运行的算术平均值与程序 B 的 100 次运行的算术平均值。然而，这样的参数化测试假设了正态（即高斯）样本分布，而通常情况下系统性能运行时直方图通常是右偏、多峰的，因此在这些情况下误用统计工具可能会产生误导性的结果。幸运的是，对于非正态分布存在更合适的统计工具，称为“非参数”测试，其示例包括 Mann-Whitney(曼-惠特尼), Anderson-Darling(安德森-达林) 和 Kolmogorov – Smirnov(科尔莫戈洛夫-斯米尔诺夫)（下一节将详细介绍）。对于那些希望自己搭建自动化性能回归测试框架的人，Python 和 R 提供了这些可下载的软件包，而像stats-pal[16]这样的开源项目提供了现成的框架，可插入现有的 CI/CD 流水线中。

在 [Daly et al., 2020] 中采用了一种更复杂的统计方法来识别性能回归。MongoDB 开发人员实施了变点分析，以识别其数据库产品演变代码库中的性能变化。根据 [Matteson & James, 2014]，变点分析是在时间顺序观察中检测分布变化的过程。MongoDB 开发人员利用了一种称为“E-Divisive means”的算法，该算法通过分层选择将时间序列划分为集群的分布变化点。他们的开源 CI 系统称为Evergreen[17]，并将此算法纳入其中以在图表上显示变点并打开 Jira 票据。有关此自动化性能测试系统的更多详细信息可以在 [Ingo & Daly, 2020] 中找到。

[Alam et al., 2019] 中提出了另一种有趣的方法。本文作者提出了`AutoPerf`，它使用硬件性能计数器（PMC，见 Section 3.9.1）来诊断修改程序中的性能回归。首先，它根据从原始程序收集的 PMC 优化分析文件数据来学习修改函数性能的分布。然后，它根据从修改后程序收集的 PMC 优化分析文件数据将性能的偏差检测为异常值。`AutoPerf`表明，这种设计可以有效地诊断一些最复杂的软件性能错误，例如隐藏在并行程序中的错误。

无论性能回归检测的底层算法如何，一个典型的 CI 系统应该自动执行以下操作：

1. 设置测试系统。
2. 运行基准测试套件。
3. 报告结果。
4. 确定性能是否发生了变化。
5. 对性能的意外变化发出警报。
6. 为人类分析结果可视化。

CI 系统应支持自动化和手动基准测试，产生可重复的结果，并为发现的性能回归创建工单。及时检测回归非常重要。首先，因为自回归发生以来合并的变更较少。这使我们可以指定负责调查回归的人员在他们转移到其他任务之前解

[14] LUCI - https://chromium.googlesource.com/chromium/src.git/+/master/docs/tour_of_luci_ui.md

[15] Student's t-test - https://en.wikipedia.org/wiki/Student's_t-test

[16] Stats-pal - https://github.com/JoeyHendricks/STATS-PAL

[17] Evergreen - https://github.com/evergreen-ci/evergreen

决问题。此外，对于开发人员来说，解决回归问题要容易得多，因为所有细节仍然新鲜在他们的脑海中，而不是在几周之后。

最后，CI 系统不应仅仅在软件性能回归上发出警报，还应在性能改进方面发出意外警报。例如，有人可能提交了一个看似无害的提交，然而，在自动性能回归测试中，它将延迟减少了惊人的 10%。您最初的直觉可能是庆祝这次幸运的性能提升并继续您的一天。然而，尽管此提交可能已通过 CI 流水线中的所有功能测试，但很有可能这个意外的延迟改进揭示了功能测试中的一个漏洞，这个漏洞只在性能回归结果中表现出来。这种情况经常发生，足以明确提及：将自动性能回归测试工具视为整体软件测试框架的一部分，而不是一个孤立的部分。

本书的作者强烈建议建立一个自动化的统计性能跟踪系统。尝试使用不同的算法，看看哪种对您的应用程序效果最好。这当然需要时间，但这将是对项目未来性能健康的坚实投资。

2.4 手动性能测试

当工程师能够在开发过程中利用现有的性能测试基础设施时，这是很棒的。在前一节中，我们讨论了 CI 系统的一个很好的功能是可以向其提交性能评估作业的可能性。如果支持这一点，那么系统将返回测试开发人员想要提交到代码库的补丁的结果。由于各种原因，这可能并不总是可能的，比如硬件不可用、设置对于测试基础设施来说过于复杂、需要收集额外的指标。在本节中，我们提供了进行本地性能评估的基本建议。

当我们在代码中进行性能改进时，我们需要一种方法来证明我们确实取得了改进。此外，当我们提交常规代码更改时，我们希望确保性能没有退化。通常，我们通过以下方式来做到这一点：1）测量基准性能，2）测量修改后程序的性能，3）将它们进行比较。在这种情况下，我们的目标是比较同一个功能程序的两个不同版本的性能。例如，我们有一个以递归方式计算斐波那契数的程序，我们决定以迭代的方式重写它。两者在功能上是正确的，并且产生相同的结果。现在我们需要比较两个程序的性能。

强烈建议不仅获取单个测量值，而是多次运行基准测试。因此，我们有 N 次基准的测量结果和 N 次修改版本的程序的测量结果。现在我们需要一种方法来比较这两组测量结果，以确定哪个更快。这个任务本身是难以解决的，有很多方法可以被测量结果所欺骗，并且可能从中得出错误的结论。如果你问任何数据科学家，他们都会告诉你，你不应该依赖于单一的度量指标（最小值/平均值/中值等）。

考虑图 4 中收集的两个程序版本的性能测量的分布。这个图显示了对于给定版本的程序，我们得到特定时间的概率。例如，版本`A`完成在大约 102 秒的概率约为 32%。诱人的是说`A`比`B`更快。然而，这只有一定的概率`P`才是真的。这是因为有一些`B`的测量比`A`快。即使在所有`B`的测量都比每个`A`的测量都慢的情况下，概率`P`也不等于`100%`。这是因为我们总是可以为`B`生成一个额外的样本，这个样本可能比一些`A`的样本更快。

使用分布图的一个有趣优势是它允许您发现基准测试的不良行为。[18]如果分布是双峰的，那么基准测试很可能经历了两种不同类型的行为。双峰分布测量的常见原因是代码具有快速路径和慢速路径，比如访问缓存（缓存命中 vs 缓存未命中）和获取锁（竞争锁 vs 非竞争锁）。要“修复”这个问题，应该将不同的功能模式分离出来并单独进行基准测试。

数据科学家通常通过绘制分布图来呈现测量结果，并避免计算加速比。这样做可以消除偏见的结论，并允许读者自行解释数据。绘制分布的一种流行方式是使用箱线图（参见图 5 ），它允许在同一图表上比较多个分布。

虽然可视化性能分布可能有助于发现某些异常，但开发人员不应该将它们用于计算加速比。一般来说，通过查看性能测量分布来估算加速比是困难的。此外，正如前一节讨论的那样，它在自动基准测试系统中不起作用。通常，我们希望获得一个标量值，该值代表两个程序版本的性能分布之间的加速比，例如，“版本`A`比版本`B`快`X%`”。

[18] 另一种检查方法是运行正态性检验: https://en.wikipedia.org/wiki/Normality_test.

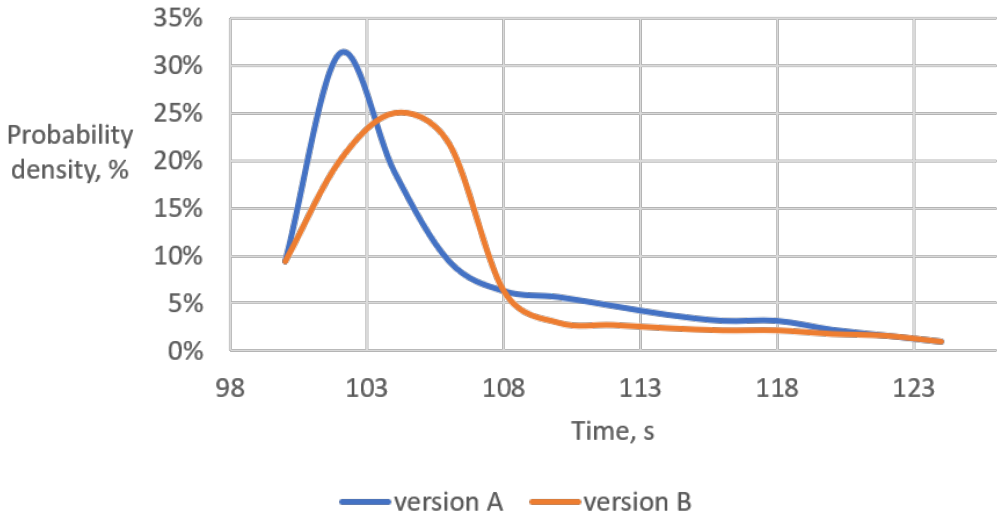


Figure 4: Comparing 2 performance measurement distributions.

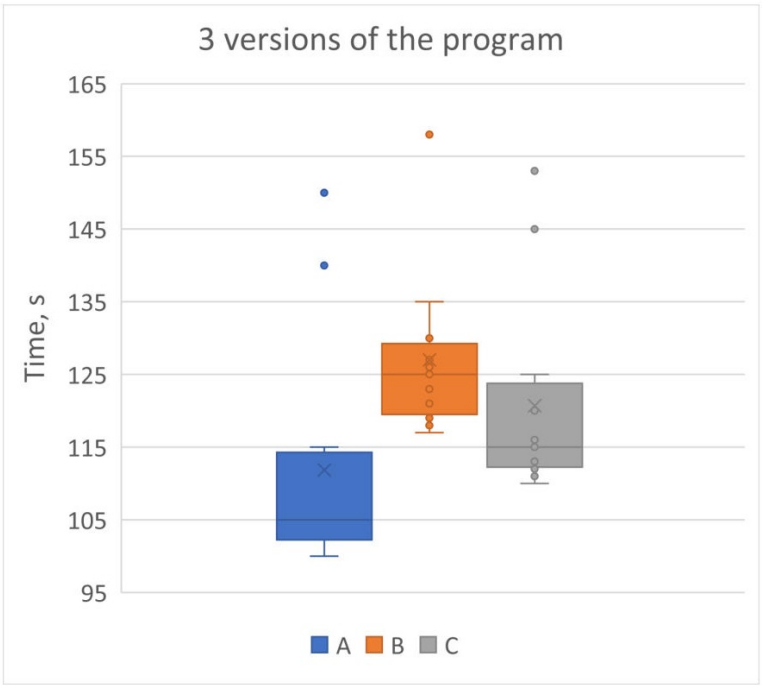


Figure 5: Box plots.

2.5 假设检验方法

假设检验方法是确定两个分布之间的统计关系的一种方法。如果根据阈值概率（显著水平），数据集之间的关系将拒绝零假设[19]，则将比较被认为是统计显著的。

- 如果分布是高斯分布（正态分布），那么使用参数假设检验（例如，学生 T 检验）来比较分布就足够了。尽管值得一提的是，性能数据中很少见到高斯分布。因此，在使用假设为高斯分布的公式时要谨慎。
- 如果要比较的分布不是高斯分布（例如，严重偏斜或多峰），那么可以使用非参数检验（例如，曼-惠特尼 U 检验[20]、克鲁斯卡尔-沃利斯单因素方差分析[21]等）。

假设检验方法非常适用于确定速度提升（或减慢）是否是随机的。因此，最好在自动化测试框架中使用它来验证提交是否引入了性能回归。关于性能工程统计学的一个很好的参考资料是 Dror G. Feitelson 的书《计算机系统性能评估的工作负载建模》[22]，该书有关于模态分布、偏度和其他相关主题的更多信息。

一旦通过假设检验确定了差异是统计显著的，那么速度提升可以计算为平均值或几何平均值之间的比率，但有一些注意事项。在少量样本中，均值和几何平均值可能会受到异常值的影响。除非分布的方差很小，否则不要仅考虑平均值。如果测量值的方差与均值相同数量级，那么平均值就不是一个代表性的指标。图 6 展示了程序的两个版本的示例。仅查看平均值（6a ），很容易认为版本A比版本B快 20%。然而，考虑到测量的方差（6b ），我们会发现情况并非总是如此。如果我们取版本A的最差分数和版本B的最佳分数，我们可以说版本B比版本A快 20%。对于正态分布，可以使用均值、标准差和标准误差的组合来衡量程序两个版本之间的速度提升。否则，对于偏斜或多峰样本，必须使用更适合基准测试的百分位数，例如，最小值、中位数、90th、95th、99th、最大值或这些的某种组合。

计算准确速度提升比率的一个最重要因素是收集丰富的样本集，即多次运行基准测试。这听起来可能很明显，但并不总是可行的。例如，一些SPEC CPU 2017 基准测试[23]在现代机器上运行超过 10 分钟。这意味着仅生成三个样本就需要 1 小时：每个程序版本 30 分钟。想象一下，如果你的测试套件中不只有一个基准测试，而是有数百个。即使将工作分配到多台机器上，收集足够统计数据也会变得非常昂贵。

如何知道需要多少样本才能达到统计上足够的分布？对这个问题的答案取决于你希望你的比较有多精确。样本之间的方差越低，你需要的样本数量就越少。标准差是一个告诉你分布中测量值的一致性的度量。可以通过动态限制基于标准差的基准迭代次数来实现自适应策略，也就是说，收集样本直到获得处于某个范围内的标准差。这种方法要求测量次数大于一次。否则，算法将在第一次运行基准测试之后停止，因为基准测试的单次运行具有 `std.dev.` 等于零。一旦标准差低于阈值，就可以停止收集测量。关于这种策略的更多细节可以在 [Akinshin, 2019, 第 4 章] 中找到。

另一个需要注意的重要事项是是否存在异常值。使用置信区间将一些样本（例如，冷启动）丢弃为异常值是可以接受的，但不要故意从测量集中丢弃不需要的样本。对于某些类型的基准测试，异常值可能是最重要的指标之一。例如，在对具有实时约束的软件进行基准测试时，99 百分位数可能非常有趣。Gil Tene 在YouTube上有一系列关于测量延迟的讲座，涵盖了这个话题。

2.6 软件和硬件定时器

为了对执行时间进行基准测试，工程师通常会使用现代平台提供的两种不同的定时器：

- **系统范围的高分辨率定时器**：这是一个系统定时器，通常实现为自从一个任意的起始日期（称为纪元）[24]以来经过的时钟周期数。这个时钟是单调的；即它总是递增的。可以通过系统调用从操作系统中获取系统时

[19] Null hypothesis - https://en.wikipedia.org/wiki/Null_hypothesis.

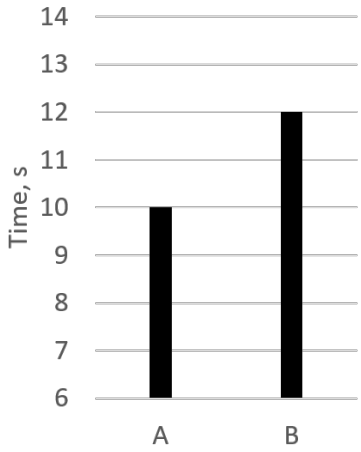
[20] Mann-Whitney U test - https://en.wikipedia.org/wiki/Mann-Whitney_U_test.

[21] Kruskal-Wallis analysis of variance - https://en.wikipedia.org/wiki/Kruskal-Wallis_one-way_analysis_of_variance.

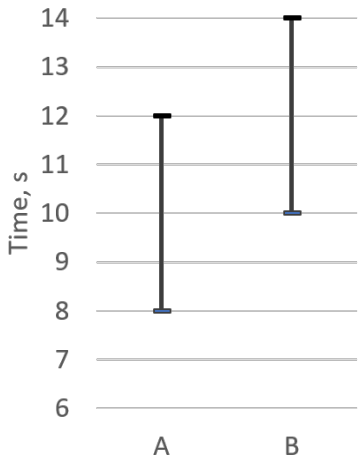
[22] 《计算机系统性能评估的工作量建模》一书 - https://www.cs.huji.ac.il/~feit/wlmod/.

[23] SPEC CPU 2017 benchmarks - http://spec.org/cpu2017/Docs/overview.html#benchmarks

[24] Unix 纪元从 1970 年 1 月 1 日 00:00:00 UT 开始：https://en.wikipedia.org/wiki/Unix_epoch。



(a) 仅平均值



(b) 完整测量间隔

Figure 6: 显示平均值可能具有误导性的两个直方图。

间。[25]在 Linux 系统上，可以通过clock_gettime系统调用来访问系统定时器。系统定时器具有纳秒级分辨率，在所有 CPU 之间保持一致，并且与 CPU 频率无关。虽然系统定时器可以返回纳秒精度的时间戳，但由于通过clock_gettime系统调用获取时间戳需要很长时间，因此不适合测量短时间内发生的事件。但是对于持续时间超过一微秒的事件来说，这是可以接受的。在 C++ 中访问系统定时器的de facto标准是使用std::chrono，如 Listing 25所示。

代码清单：使用 C++ std::chrono 访问系统定时器

```
#include <cstdint>
#include <chrono>

// 返回经过的纳秒数
uint64_t timeWithChrono() {
  using namespace std::chrono;
  auto start = steady_clock::now();
  // 运行一些代码
  auto end = steady_clock::now();
  uint64_t delta = duration_cast<nanoseconds>
      (end - start).count();
  return delta;
}
```

- **时间戳计数器（TSC）**：这是一种硬件定时器，实现为硬件寄存器。TSC 是单调的，并且具有恒定的速率，即不考虑频率变化。每个 CPU 都有自己的 TSC，它只是已经过去的参考周期数（参见 Section 4.6）。适用于持续时间从纳秒到一分钟的短事件的测量。可以使用编译器内置函数__rdtsc来获取 TSC 的值，如 Listing 25所示，该函数在底层使用RDTSC汇编指令。有关使用RDTSC汇编指令对代码进行基准测试的更低级别的详细信息，可以参考白皮书 [Paoloni, 2010]。

  代码清单：使用 __rdtsc 编译器内置函数访问 TSC

```
#include <x86intrin.h>
#include <cstdint>

// 返回经过的参考时钟数
uint64_t timeWithTSC() {
    uint64_t start = __rdtsc();
    // 运行一些代码
    return __rdtsc() - start;
}
```

选择使用哪种定时器非常简单，取决于您要测量的事物持续的时间有多长。如果您要测量的时间很短，TSC 会给出更高的精度。相反，使用 TSC 来测量运行数小时的程序是没有意义的。除非您确实需要周期精度，否则系统定时器应该足够用于大多数情况。要记住的重要一点是，访问系统定时器通常的延迟比访问 TSC 要高。进行clock_gettime系统调用可能比执行RDTSC指令慢十倍以上，后者需要 20 多个 CPU 周期。这在尤其需要最小化测量开销时可能变得很重要，尤其是在生产环境中。有关在各种平台上访问定时器的不同 API 的性能比较可在 CppPerformanceBenchmarks 存储库的wiki 页面[26]上找到。

[25] 检索系统时间 - https://en.wikipedia.org/wiki/System_time#Retrieving_system_time

[26] CppPerformanceBenchmarks wiki - https://gitlab.com/chriscox/CppPerformanceBenchmarks/-/wikis/ClockTimeAnalysis)

2.7 微基准测试

微基准测试是人们编写的小型独立程序，用于快速测试假设。通常，微基准测试用于选择某个相对较小算法或功能的最佳实现。几乎所有现代语言都有基准测试框架。在 C++ 中，可以使用 Google 的benchmark[27]库，C# 有BenchmarkDotNet[28]库，Julia 有BenchmarkTools[29]包，Java 有JMH[30]（Java Microbenchmark Harness），等等。

在编写微基准测试时，非常重要的一点是确保您要测试的场景在运行时确实被微基准测试执行。优化编译器可能会消除可能使实验无效的重要代码，甚至更糟的是，导致您得出错误的结论。在下面的示例中，现代编译器很可能会消除整个循环：

```
// foo不会对字符串创建进行基准测试
void foo() {
  for (int i = 0; i < 1000; i++)
    std::string s("hi");
}
```

测试这一点的一种简单方法是检查基准测试的性能概要，并查看预期的代码是否突出显示为热点。有时可以立即发现异常的计时情况，因此在分析和比较基准测试运行时，请务必运用常识。防止编译器消除重要代码的一种流行方法是使用类似DoNotOptimize[31]的辅助函数，其在底层执行必要的内联汇编操作：

```
// foo对字符串创建进行基准测试
void foo() {
  for (int i = 0; i < 1000; i++) {
    std::string s("hi");
    DoNotOptimize(s);
  }
}
```

如果编写得当，微基准测试可以成为性能数据的良好来源。它们经常用于比较关键函数的不同实现的性能。一个好的基准测试的定义在于它是否在功能实际使用的真实条件下测试性能。如果基准测试使用与实践中给定的输入不同的合成输入，则该基准测试很可能会误导您，并导致您得出错误的结论。此外，当基准测试在没有其他要求的进程的系统上运行时，它具有所有可用的资源，包括 DRAM 和缓存空间。这样的基准测试可能会成为更快版本功能的冠军，即使它消耗的内存比其他版本多。但是，如果有占用 DRAM 的邻近进程，导致基准测试进程的内存区域被换出到磁盘，结果可能相反。

出于同样的原因，当从对函数进行单元测试中得出结果时，请谨慎。现代单元测试框架，例如 GoogleTest，提供每个测试的持续时间。但是，这些信息不能替代一个精心编写的基准测试，该测试使用真实的输入在实际条件下测试函数（有关更多信息，请参见 [Fog, 2004, 第 16.2 章]）。在实践中无法复制确切的输入和环境，但这是开发人员在编写良好基准测试时应考虑的事项。

问题和练习

1. 对一系列测量结果取平均时间安全吗？
2. 假设你已经发现了一个性能漏洞，你现在试图在你的开发环境中修复它。你将如何减少系统中的噪音，以获得更明显 的基准测试结果？

[27] Google benchmark 库 - https://github.com/google/benchmark

[28] BenchmarkDotNet - https://github.com/dotnet/BenchmarkDotNet

[29] Julia BenchmarkTools - https://github.com/JuliaCI/BenchmarkTools.jl

[30] Java Microbenchmark Harness - http://openjdk.java.net/projects/code-tools/jmh/etc

[31] 对于 JMH，这被称为`Blackhole.consume()`。

3. 使用函数级别的单元测试跟踪程序的整体性能可以吗?
4. 您的组织是否有性能回归系统?如果有，可以改进吗?如果没有，请考虑安装一个的策略。请考虑以下因素:什么是变化的，什么是没有变化的（源代码、编译器、硬件配置等），更改发生的频率是多少，测量方差是多少，基准测试的运行时间是多少，您可以运行多少次迭代?

AI 回答:

1. **对一系列测量结果取平均时间安全吗?** 取平均时间是一种常见的方法来减少随机误差并提供性能测量的估计值。然而，这种方法并不总是安全的，因为它可能掩盖了性能数据中的某些模式或异常。例如，如果测量中存在系统性偏差或者数据分布不是正态分布，仅仅取平均值可能不足以提供准确的性能评估。在这种情况下，可能需要使用更复杂的统计方法来分析数据，如中位数、百分位数或非参数统计测试。

2. **如何减少系统中的噪音，以获得更明显的基准测试结果?** 为了减少噪音并提高基准测试的准确性，可以采取以下措施:

   - **控制环境变量**：确保在相同的硬件和软件配置下运行测试，包括相同的操作系统版本、编译器设置和硬件状态。
   - **消除外部干扰**：在测试期间关闭不必要的应用程序和服务，以减少对 CPU、内存和 I/O 资源的竞争。
   - **预热系统**：在正式测试前运行一段时间的基准测试，以确保系统资源（如 CPU 缓存）处于活跃状态。
   - **多次运行测试**：进行多次测试并使用统计方法（如置信区间）来分析结果，以减少随机误差的影响。
   - **使用专用工具**：使用专门设计来减少噪音的工具，如 tempci，来帮助隔离和控制测试环境。

3. **使用函数级别的单元测试跟踪程序的整体性能可以吗?** 单元测试通常用于验证单个函数或模块的正确性，而不是用于跟踪整体性能。单元测试可能无法捕捉到整体性能问题，因为它们通常不模拟真实的运行时环境，也不考虑与其他系统组件的交互。为了跟踪整体性能，需要进行更全面的基准测试，这些测试应该在接近生产环境的条件下运行，并考虑所有相关的系统组件。

4. **组织是否有性能回归系统?** 这个问题的答案取决于具体的组织。如果组织已经有性能回归系统，那么可以考虑以下因素来评估和改进系统:

   - **变化的因素**：确定源代码、编译器、硬件配置等是否有变化，以及这些变化如何影响性能。
   - **更改频率**：评估代码更改的频率，以及这些更改对性能的潜在影响。
   - **测量方差**：分析性能测试结果的方差，以确定是否需要更多的迭代或更精细的控制来减少噪音。
   - **基准测试运行时间**：考虑基准测试的运行时间是否足够长，以确保结果的准确性和可靠性。
   - **迭代次数**：确定可以运行多少次迭代，以及如何平衡测试的频率和资源消耗。

   如果没有性能回归系统，可以考虑安装一个，同时考虑上述因素来设计一个适合组织需求的系统。这可能包括选择合适的工具、定义性能指标、设置阈值以及集成到现有的开发和部署流程中。

章节总结

- 调试性能问题通常比调试功能性 bug 更困难，这是因为测量的不稳定性。
- 除非设定特定的目标，否则不能停止优化。要知道是否达到了预期的目标，您需要制定有意义的定义和衡量标准。根据您关心的内容，可以是吞吐量、延迟、每秒操作数（性能屋顶）等。
- 现代系统具有非确定性性能。消除系统中的非确定性对于定义明确、稳定的性能测试（例如微基准测试）非常有帮助。在生产部署中测量性能需要使用统计方法来分析结果，以应对嘈杂的环境。
- 越来越多地，大型分布式软件的供应商选择在生产系统上直接对性能进行配置和监控，这需要使用轻量级的配置技术。
- 为了防止性能回归泄漏到生产软件中，采用自动化的统计性能跟踪系统非常有益。这种持续集成系统应该运行自动化性能测试，可视化结果，并在发现性能异常时发出警报。
- 可视化性能分布还可以帮助发现性能异常。这是向广大受众展示性能结果的安全方式。

- 使用假设检验方法确定性能分布之间的统计关系。一旦确定差异在统计上是显著的，就可以计算加速比，即均值或几何均值之比。
- 可以放弃冷运行以确保一切运行良好，但不要故意放弃不想要的数据。如果决定放弃某些样本，应对所有分布均匀地进行。
- 为了测量执行时间，工程师可以使用现代平台提供的两种不同的定时器。系统范围内的高分辨率定时器适用于测量持续时间超过一微秒的事件。要以高精度测量短期事件，使用时间戳计数器。
- 微基准测试对于快速证明某些内容非常有用，但您应该始终在实际条件下的真实应用程序上验证自己的想法。确保您正在对有意义的代码进行基准测试，方法是检查性能分析。